

Multi-Agent 系统与 CrewAI 框架详解

—— 从概念到工程实践

2026 年 5 月

第一章 Multi-Agent 系统简介

Multi-Agent(多智能体)系统是指由多个能够自主决策、相互协作或竞争的智能体(Agent)共同组成的系统。每个 Agent 都有自己的目标、能力和决策逻辑,通过彼此交互来完成单个 Agent 难以胜任的复杂任务。

1.1 核心概念

Agent(智能体) 是能够感知环境、做出决策并采取行动的实体。在 LLM 时代,一个 Agent 通常由“大模型 + 工具调用能力 + 记忆 + 角色设定”构成。

Multi-Agent 则是把多个这样的 Agent 组织起来,让它们分工协作。比如一个 Agent 负责规划,另一个负责编码,第三个负责审查测试。

1.2 典型架构模式

常见的组织方式有几种:

- **层级式(Hierarchical):** 由一个主控 Agent(Orchestrator / Supervisor)分派任务给下属 Agent,类似项目经理带团队,Anthropic 自己的 Research 系统就是典型代表。
- **对等协作式(Peer-to-Peer):** 各 Agent 地位平等,通过消息传递自主沟通。
- **流水线式(Pipeline):** Agent 按固定顺序传递中间结果,前一个的输出是后一个的输入。
- **辩论/投票式(Debate):** 多个 Agent 对同一问题给出独立答案,再通过讨论或投票收敛结果,用于提升推理可靠性。

1.3 为什么需要 Multi-Agent

单个 LLM Agent 在面对超长上下文、多领域知识、并行任务时容易力不从心。多 Agent 的优势主要体现在:

- **关注点分离:** 每个 Agent 只需精通自己的角色,提示词更聚焦。
- **并行加速:** 独立子任务可同时推进。
- **上下文隔离:** 避免单个上下文窗口被无关信息污染。
- **专业化分工:** 可以用不同模型、不同工具组合扮演不同角色。

1.4 常见框架概览

业界目前比较流行的多 Agent 框架包括:

- **LangGraph** —— 基于图的状态机编排。
- **AutoGen** —— 微软出品,擅长对话式协作。
- **CrewAI** —— 角色驱动,API 简洁。
- **MetaGPT** —— 模拟软件公司分工。
- **Claude Agent SDK** —— Anthropic 自家方案。

1.5 实践中的挑战

Multi-Agent 也不是银弹。Anthropic 在工程博客里提到过几个常见坑:

- **Token 成本** 会成倍上升(多 Agent 通常比单 Agent 多消耗 4 倍以上 token)。
- **错误会累积放大**,一个 Agent 的幻觉可能误导整条链。
- **协调开销** —— Agent 之间的通信和对齐本身就需要设计。
- **评估困难** —— 如何判断整个系统而不只是单个 Agent 的表现。

经验上的建议是:只有当任务确实需要并行或多视角时才上 Multi-Agent,简单任务用单 Agent + 好的工具调用往往更高效、更便宜、更可靠。

第二章 主流 Multi-Agent 框架对比

按目前(截至 2026 年初)的社区热度和企业采用情况,主流框架可以分为三个梯队。

2.1 第一梯队:LangGraph

LangGraph 是目前最流行的多 Agent 框架,几乎是事实上的工业标准。它脱胎于 LangChain 生态,基于“状态图”(StateGraph)的编排方式,既支持简单的链式流程,也支持复杂的循环、分支、人在环路(human-in-the-loop)。GitHub star 增长非常快,LangChain 团队也在持续推它作为主力产品。企业落地案例最多,包括 Klarna、LinkedIn、Uber 等都在用。

优点: 灵活、可调试、生态完善、有专门的可视化工具 LangGraph Studio。

缺点: 学习曲线略陡,需要理解状态机思维。

2.2 第二梯队:CrewAI 和 AutoGen

CrewAI 在过去一年增长最猛,以“角色 + 任务”为核心抽象,API 极其简洁,几十行代码就能搭出一个像模像样的协作团队。特别受非资深开发者和快速原型场景欢迎,GitHub star 数已经追上甚至超过 AutoGen。

AutoGen 是微软研究院出品,学术界影响力大,擅长对话驱动的多 Agent 交互。微软在 2024 年推出了重写版 AutoGen 0.4,架构更现代,但生态分裂也让一部分用户流失。在企业内部(尤其是微软系)仍有稳固地位。

2.3 第三梯队:专业化框架

- **MetaGPT** 主打“模拟软件公司”,在代码生成场景很有特色,中文社区热度高。
- **OpenAI Swarm**(及其继任者 Agents SDK)轻量、官方背书,适合 OpenAI 生态深度用户。

- **Claude Agent SDK** 是 Anthropic 自家的方案,在 Claude 模型上调优最好,适合做生产级 Agent。

2.4 选型建议

简明的选型决策表如下:

场景	推荐框架
认真做项目、追求长期可维护性	LangGraph
快速验证想法、原型阶段	CrewAI
重度使用 Claude 模型	Claude Agent SDK
微软 Azure 生态内	AutoGen
代码生成、模拟开发团队	MetaGPT
OpenAI 生态深度用户	OpenAI Agents SDK

第三章 CrewAI 程序示例

3.1 示例场景

下面是一个研究 + 写作团队的完整例子,两个 Agent 协作完成一篇技术简报:一个负责调研,一个负责撰写。

3.2 完整代码

```
"""
CrewAI 示例:研究 + 写作协作团队
任务:针对某个技术话题,先调研、再撰写一篇简报

依赖安装:
    pip install crewai crewai-tools

环境变量(任选一种 LLM):
    export OPENAI_API_KEY="sk-..."
    # 或使用 Anthropic Claude:
    # export ANTHROPIC_API_KEY="sk-ant-..."
"""

import os
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool # 联网搜索工具(需要 SERPER_API_KEY)

# ===== 1. 准备工具 =====
# SerperDevTool 用于 Google 搜索,需要在 serper.dev 申请免费 key
search_tool = SerperDevTool()

# ===== 2. 定义 Agent(智能体) =====
# 每个 Agent 都有清晰的角色、目标和背景故事 — 这是 CrewAI 的核心思想

researcher = Agent(
    role="资深技术研究员",
    goal="搜集关于 {topic} 的最新、最权威的技术资料,并提炼关键信息",
    backstory=(
        "你是一名拥有十年经验的技术研究员,擅长快速定位高质量信息源,"
```

```
        "能够从大量噪声中筛选出真正有价值的内容。你做事严谨,引用必有出处。"
    ),
    tools=[search_tool],
    verbose=True,          # 打印思考过程,方便调试
    allow_delegation=False, # 不允许把任务转派给别人
)
```

```
writer = Agent(
    role="技术内容撰稿人",
    goal="基于研究员提供的资料,撰写一篇结构清晰、易读的技术简报",
    backstory=(
        "你是一名技术博客作者,擅长把复杂概念讲得通俗易懂。"
        "你的文章注重逻辑结构,语言简洁,从不堆砌术语。"
    ),
    tools=[],              # 撰稿人不需要搜索,只用研究员的产出
    verbose=True,
    allow_delegation=False,
)
```

```
# ===== 3. 定义 Task(任务) =====
# Task 描述要做什么、期望产出什么、由哪个 Agent 来做
```

```
research_task = Task(
    description=(
        "针对话题「{topic}」进行深入调研,要求:\n"
        "1. 找到至少 5 个权威信息来源(官方文档、知名博客、论文等)\n"
        "2. 总结当前的技术现状、主流方案、关键挑战\n"
        "3. 列出每个信息源的链接和核心观点"
    ),
    expected_output=(
        "一份结构化的调研笔记,包含:\n"
        "- 话题背景概述(约 200 字)\n"
        "- 5 个以上信息源 + 核心观点摘要\n"
        "- 3 个值得深入展开的子话题"
    ),
    agent=researcher,
)
```

```
writing_task = Task(
    description=(
        "基于研究员的调研笔记,撰写一篇面向技术读者的简报。"
        "要求:\n"
        "1. 800-1200 字"
    )
)
```

```
"2. 包含引言、核心内容、总结三部分\n"
"3. 在引用观点时注明来源\n"
"4. 避免堆砌术语,确保非专家也能读懂"
),
expected_output="一篇 Markdown 格式的技术简报",
agent=writer,
context=[research_task], # 关键:把研究任务的产出作为本任务的上下文
output_file="briefing.md", # 自动保存到文件
)

# ===== 4. 组建 Crew(团队) =====
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential, # 顺序执行:先调研,再写作
    verbose=True,
)

# ===== 5. 启动!=====
if __name__ == "__main__":
    result = crew.kickoff(
        inputs={"topic": "Multi-Agent 系统在 2026 年的工程实践"}
    )
    print("\n===== 最终产出 =====\n")
    print(result)
```

3.3 核心概念讲解

这个例子展示了 CrewAI 的四个核心概念:

Agent(智能体) 是 CrewAI 最有特色的设计。每个 Agent 需要定义三件事:role(角色)、goal(目标)、backstory(背景故事)。背景故事不是装饰 —— 它会被注入到 prompt 里,实质性地影响 Agent 的行为风格。这个设计让 Agent 的角色感非常强,这也是为什么 CrewAI 在快速原型场景特别受欢迎。

Task(任务) 描述具体要做什么。description 是任务说明,expected_output 描述期望产出的格式 —— 这个字段很重要,它帮助 Agent 理解何时算“完成”。context=[research_task] 是关键

的一行,它把前一个任务的产出自动传递给当前任务,这就是 Agent 之间“协作”的实现方式。

Crew(团队) 把 Agent 和 Task 组装起来。Process.sequential 表示顺序执行;还可以用 Process.hierarchical,让一个 Manager Agent 自动分派任务,更接近真实公司的运作方式。

kickoff() 启动整个流程,inputs 里的变量会替换掉 description 中的 {topic} 占位符。

3.4 运行准备与预期输出

运行起来需要做的准备:

- 安装包: `pip install crewai crewai-tools`
- 准备 API key: OpenAI 或 Anthropic 的任一个,加上 serper.dev 的免费搜索 key(每月 2500 次免费额度)。
- 如果想用 Claude 而不是 GPT,在 Agent 里加一行 `llm="anthropic/claude-sonnet-4-5"` 就行。

运行后,终端里能看到两个 Agent 的“思考过程”:研究员先做几次搜索、整理资料,然后撰稿人接过笔记开始写作。最后会在当前目录生成一个 briefing.md 文件。整个过程大概几十秒到一两分钟,token 消耗大约几千到一万。

第四章 CrewAI 的运作机制

要真正理解 CrewAI,需要把它拆成两个层面来看:外层的协作编排(Agent 之间怎么配合),和内层的 LLM 调用(单个 Agent 怎么思考和行动)。下面按从外到内的顺序讲。

4.1 整体执行流程

`crew.kickoff(inputs={...})` 调用后,CrewAI 内部大致发生这些事:

第一步:输入注入与任务准备。 Crew 会遍历所有 Task,把 inputs 字典里的变量(比如 {topic})填入 description 和 expected_output 字符串里。这是简单的模板替换,不涉及 LLM。

第二步:按 Process 调度任务。 如果是 Process.sequential,任务按列表顺序一个个执行;如果是 Process.hierarchical,CrewAI 会自动创建一个 Manager Agent,由它来决定每一步派给谁、何时结束。

第三步:任务交给对应的 Agent 执行。 这一步才真正进入 LLM 调用环节。

第四步:产出传递与最终汇总。 Task 完成后,产出会根据 context 配置传递给后续任务,最终 kickoff() 返回最后一个任务的输出(或一个汇总的 CrewOutput 对象)。

4.2 单个 Agent 内部的 ReAct 循环

这是核心部分。一个 Agent 拿到 Task 之后,并不是只调用一次 LLM,而是进入一个叫

ReAct(Reasoning + Acting)的循环。

第一次 LLM 调用时,CrewAI 把 Agent 的 role、goal、backstory、Task 的 description 和 expected_output、可用工具列表、以及上游任务的产出,全部拼成一个大 prompt 发给 LLM。

LLM 返回一段包含“思考 + 行动”的文本,比如:

```
Thought: 我需要先搜索关于这个话题的最新资料
Action: search_tool
```

```
Action Input: {"query": "multi-agent systems 2026"}
```

CrewAI 解析出 Action 和 Action Input,在本地执行对应的工具(这里就是调用 Serper 搜索 API),

拿到结果后,把结果作为 Observation 拼接到对话历史里,再次调用 LLM:

```
Thought: ...(上一轮)
Action: search_tool
Action Input: ...
Observation: [搜索返回的内容]
```

LLM 看到搜索结果后,可能继续搜索、可能开始整理答案、也可能直接给出最终结论。这个“思考 → 行动 → 观察”的循环会一直持续,直到 LLM 输出 Final Answer,或者达到 max_iter(默认 25 次)上限。

最终的 Final Answer 就是这个 Task 的产出,会传给下一个 Task 或返回给用户。

4.3 LLM 调用的具体实现

CrewAI 底层用 **LiteLLM** 这个库统一处理对 LLM 的调用。LiteLLM 是个适配层,把不同厂商 (OpenAI、Anthropic、Google、本地 Ollama 等)的 API 差异屏蔽掉,对外暴露统一的接口。这就是为什么你只要换一个 llm= 参数,就能在 GPT、Claude、Gemini 之间切换。

模型配置有三个层级,优先级从高到低:

- **Agent 级别:** Agent(..., llm="...")
- **Crew 级别:** Crew(..., manager_llm="...")
- **环境变量:** 读 OPENAI_API_KEY 等。如果都不设,默认用 gpt-4o-mini。

举几个具体写法:

```
from crewai import LLM

# 方式 1:直接传字符串(LiteLLM 格式)
agent = Agent(role="...", llm="anthropic/claude-sonnet-4-5")

# 方式 2:用 LLM 类,可以细调参数
```

```
agent = Agent(  
    role="...",  
    llm=LLM(  
        model="anthropic/claude-sonnet-4-5",  
        temperature=0.3,  
        max_tokens=4000,  
    )  
)  
  
# 方式 3:本地模型(Ollama)  
agent = Agent(role="...", llm="ollama/llama3.1")
```

4.4 Prompt 组装机制

CrewAI 内部有一套 prompt 模板,大致结构是:

System prompt 包含 Agent 的 role、goal、backstory,以及关于 ReAct 输出格式的说明(教 LLM 怎么写 Thought / Action / Observation)。

User prompt 包含 Task 的 description、expected_output、上下文(来自 context=[...] 的上游产出)、可用工具的描述。

每一轮 LLM 调用,CrewAI 还会把之前的“思考-行动-观察”历史拼接进去,所以对话历史是单调增长的——这也是为什么多 Agent 系统的 token 消耗远高于单次调用,Anthropic 的研究博客里提到过通常是 4 倍以上,在 CrewAI 这种 ReAct 范式里也类似。

4.5 工具调用的两种实现

CrewAI 支持两种工具调用模式:

Function Calling 模式(推荐): 如果用 GPT-4、Claude 这类支持原生 function calling 的模型,CrewAI 会把工具转成 OpenAI function schema,由模型原生返回结构化的 tool_call。这种方式更稳定,解析不容易出错。

ReAct 文本模式(兼容老模型): 对不支持 function calling 的模型(比如某些开源模型),CrewAI 退

化到纯文本的 ReAct 协议,靠正则解析 Action: 和 Action Input: 这样的字符串。这种方式更脆弱,LLM 偶尔会输出格式错误。

4.6 层级模式下的特殊机制

如果用 `Process.hierarchical`,CrewAI 会自动创建一个 Manager Agent(你需要通过 `manager_llm` 指定它用哪个模型,通常用更强的模型比如 GPT-4 或 Claude Opus)。Manager 不执行具体任务,它的工作是:

读取所有 Task → 决定派给哪个下属 Agent → 用一个叫 “Delegate work to coworker” 的内置工具发起委派 → 下属 Agent 完成后,Manager 评估结果决定下一步。这本质上是把 “调度” 也变成了一次 LLM 调用,所以层级模式更灵活但也更贵。

4.7 完整调用图景

把上面所有环节串起来,以前面的研究员 + 撰稿人例子来说,实际发生的事情是:

kickoff() 被调用 → `research_task` 启动 → 研究员 Agent 进入 ReAct 循环 → 第 1 次调 LLM,LLM 决定搜索 → CrewAI 调用 Serper API → 第 2 次调 LLM 带着搜索结果 → 可能再搜几次 → 第 N 次调 LLM 输出 Final Answer → `research_task` 完成 → `writing_task` 启动,带着上一步的产出作为 context → 撰稿人 Agent 进入 ReAct 循环(它没工具,所以通常一次就出 Final Answer) → 写入 `briefing.md` → 返回结果。

整个过程通常是 5-15 次 LLM 调用,具体数量取决于研究员搜索了几次。

第五章 CrewAI 运作流程图解

本章用三张结构化的流程描述来可视化前面讲到的机制。

5.1 整体执行流程图

从 kickoff 到产出的端到端流程:

步骤	内容	是否消耗 LLM
入口	用户调用 kickoff(inputs)	否
1	输入注入:把 inputs 填入 task 模板	否
2	按 Process 调度(sequential 或 hierarchical)	hierarchical 是
3	Agent 执行 Task,进入 ReAct 循环	是(多次)
4	产出传递:通过 context 传给下一个 task	否
循环	若还有任务,回到第 3 步	—
产出	CrewOutput 返回最终任务的产出	否

第 3 步是真正花时间和 token 的地方,因为 Agent 在内部要跑 ReAct 循环。

5.2 Agent 内部的 ReAct 循环

一个 Agent 拿到 Task 后,内部会按以下循环执行:

环节	动作
Prompt 组装	role + goal + backstory + task + 工具列表 +

环节	动作
	历史 拼成大 prompt
调用 LLM	通过 LiteLLM 统一接口,调 GPT / Claude / Gemini 等
LLM 返回	Thought + Action,或直接 Final Answer
解析输出	CrewAI 解析:是 Action 还是 Final Answer?
Action 分支	执行工具(如 Serper 搜索),本地代码调用 API
Observation	把工具结果拼回历史,准备下一轮 LLM 调用
Final 分支	输出 Task 产出,退出本 Agent

循环最多 25 次(默认 max_iter),每一轮都会调用一次 LLM。

5.3 两个 Agent 协作的时序

以第三章的研究员 + 撰稿人为例,实际跑起来的完整时序如下:

阶段	执行者	动作	类型
1	研究员	LLM 调用 1:决定搜索关键词	LLM
2	研究员	工具调用:Serper 搜索,返回结果	Tool
3	研究员	LLM 调用 2:分析结果,决定再搜	LLM
4	研究员	工具调用:再次搜索,补充资料	Tool
5	研究员	LLM 调用 3:输出 Final Answer(调研笔记)	LLM

阶段	执行者	动作	类型
6	—	context 传递:笔记成为撰稿人 prompt 的一部分	传递
7	撰稿人	接收 context	—
8	撰稿人	LLM 调用 4:直接撰写, 无需工具	LLM
9	撰稿人	输出 Final Answer,写入 briefing.md	—

本次共 4 次 LLM 调用 + 2 次工具调用,顺序执行。值得注意的几个细节:

- **研究员有工具,所以循环次数多。** 它要先想搜什么、搜了之后看看够不够、可能再搜一次,最后才整理输出。
- **撰稿人没有工具,通常一次 LLM 调用就出结果。** 它的输入已经包含了研究员的完整笔记,所以不需要再搜任何东西。
- **context 传递是关键机制。** 它不是两个 Agent 在“对话”,而是前者的产出被注入后者的 prompt。

第六章 Prompt 拼装与 Token 消耗深度解析

理解 prompt 拼装对调试和省 token 至关重要。本章从三个角度展开:单次调用的 prompt 结构、多轮循环里 prompt 怎么膨胀、token 消耗的实际增长曲线。

6.1 单次 LLM 调用的 prompt 分层结构

一次 LLM 调用的 prompt 拆成两大块:

层级	块	内容	是否变化
System Prompt	Agent role	如 “资深技术研究员”	固定
System Prompt	Agent goal	如 “搜集权威资料”	固定
System Prompt	Agent backstory	如 “十年经验、引用必有出处”	固定
System Prompt	ReAct 格式说明	Thought / Action / Final Answer 模板	固定
User Prompt	Task description	如 “针对话题进行深入调研”	固定
User Prompt	Expected output	如 “结构化笔记 + 5 个来源”	固定
User Prompt	Context	上游 Task 产出(第一个任务为空)	固定
User Prompt	Tools 描述	每个工具的名字 + 参数 schema	固定
User Prompt	历史对话	之前所有 Thought/Action/Observation	每轮增长

左边 System Prompt 是固定的 —— Agent 的身份和 ReAct 格式说明,整个 Task 执行期间都不会变。右边 User Prompt 是动态的 —— 尤其是底部的 “历史对话” 块,每轮都会变胖。

值得特别注意的是 ReAct 格式说明。这部分是 CrewAI 内置的 prompt 模板,大概几百个 token,

内容是告诉 LLM: “你必须按 Thought: / Action: / Action Input: / Observation: 这个格式输出”。如果用的是支持原生 function calling 的模型(GPT-4、Claude),这部分会简化;如果是普通模型,这部分会很长。

6.2 多轮循环里 prompt 是怎么膨胀的

研究员 Agent 的三轮 LLM 调用,每轮 prompt 包含的内容如下:

轮次	包含内容	估算 tokens
第 1 轮	System + Task + Tools	约 1.5K
第 2 轮	System + Task + Tools + 第 1 轮的 Thought/Action/Observation	约 3K
第 3 轮	System + Task + Tools + 第 1 轮 T/A/O + 第 2 轮 T/A/O	约 5K

这揭示了 ReAct 范式的根本成本结构。关键洞察是:

LLM 是无状态的。 每次 API 调用都得把所有上下文重发一遍 —— LLM 本身不“记得”上一轮发生了什么。所以历史块每轮都被重复传输,这是“输入 token 累积”的本质原因。

第 N 轮的输入约等于前 N-1 轮的所有产出之和。 如果一个 Task 跑 10 轮 ReAct,第 10 轮的输入可能已经达到几万 token。这就是为什么 Anthropic 的多 Agent 研究里提到 token 消耗轻松是单次调用的 4-15 倍。

Prompt caching 能救命。 Anthropic 和 OpenAI 都提供 prompt caching:固定不变的前缀 (System Prompt + Task 定义)可以被缓存,只对增量部分计费,实际成本可以降低 50-90%。

CrewAI 通过 LiteLLM 间接支持这个特性。

6.3 三种调用模式的 token 消耗对比

三种使用模式的 token 成本差距如下:

模式	LLM 调用次数	累积 tokens(示意)	适用场景
直接问 LLM	1 次	约 2K	简单事实性问答
单 Agent ReAct	3-5 次	约 15K	需要工具调用的中等任务
多 Agent 协作	8+ 次	约 40K	复杂、多角色任务

多 Agent 不只是“调用次数多”,还有跨 Agent 的 context 传递——研究员的完整产出会成为撰稿人 prompt 的一部分,叠加效应被放大了。

6.4 实战省 Token 优化建议

理解了 prompt 拼装,优化方向就清晰了:

- **让 backstory 精简但有特色**——别写小说,几句话足矣,因为它每轮都重发。
- **限定 max_iter**——默认 25 次迭代太多,实际任务很少需要超过 5-8 次,可以设 `Agent(..., max_iter=8)`。
- **只给 Agent 真正需要的工具**——工具描述也每轮重发,5 个无关工具是纯浪费。
- **开启 prompt caching**——用 Claude 模型时,长 System Prompt 自动缓存,实际成本砍掉大半。
- **优先选 sequential 而非 hierarchical**——hierarchical 模式 Manager Agent 会额外消耗 token 做调度决策。
- **用更便宜的模型跑简单子任务**——CrewAI 支持每个 Agent 用不同模型,研究类用 GPT-4o,撰写类用 Haiku 就行。



Multi-Agent 系统与 CrewAI 框架详解

理解了 CrewAI 的四层结构 —— 外层任务编排、内层 ReAct 循环、底层 LiteLLM 适配、粘合剂 prompt 拼装 —— 你就能预判它的行为、调试它的问题、优化它的成本。